

Disjoint Sets Data Structure (Chap. 21)

- A disjoint-set is a collection $S = \{S_1, S_2, \dots, S_k\}$ of distinct dynamic sets.
- Each set is identified by a member of the set, called *representative*.
- Disjoint set operations:
 - MAKE-SET(x): create a new set with only x . assume x is not already in some other set.
 - UNION(x, y): combine the two sets containing x and y into one new set. A new representative is selected.
 - FIND-SET(x): return the representative of the set containing x .

An Application of Disjoint-Set

- Determine the connected components of an undirected graph.

CONNECTED-COMPONENTS(G)

1. **for** each vertex $v \in V[G]$
2. **do** MAKE-SET(v)
3. **for** each edge $(u,v) \in E[G]$
4. **do if** FIND-SET(u) $\neq$ FIND-SET(v)
5. **then** UNION(u,v)

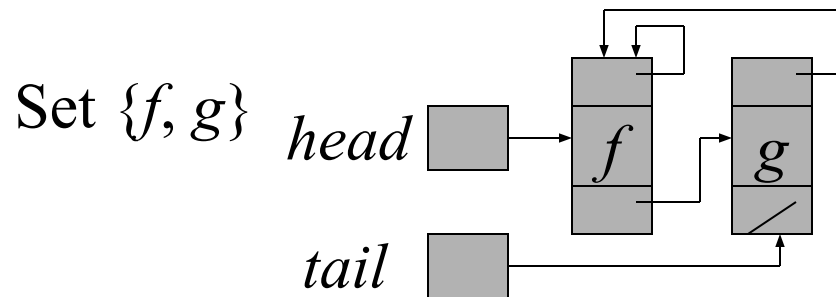
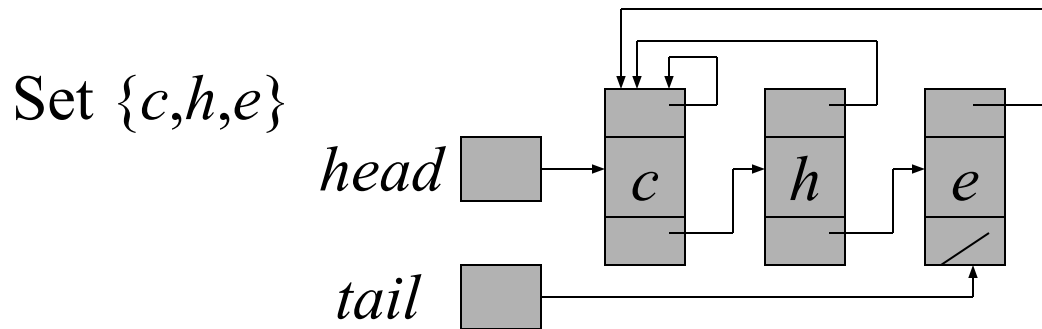
SAME-COMPONENT(u,v)

1. **if** FIND-SET(u)=FIND-SET(v)
2. **then return** TRUE
3. **else return** FALSE

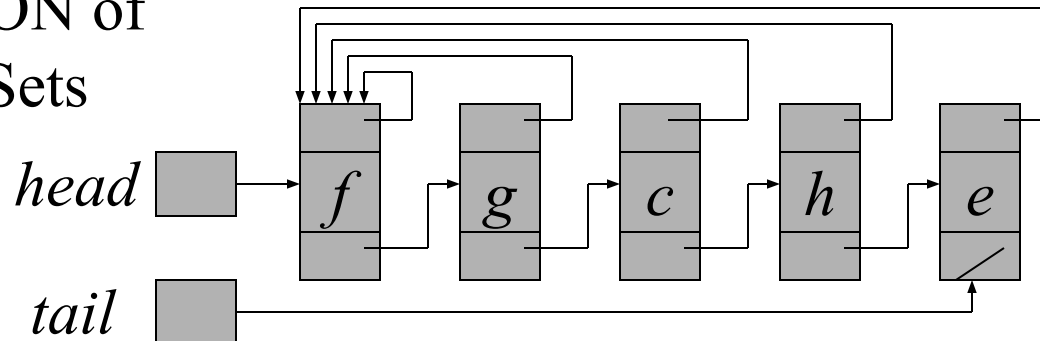
Linked-List Implementation

- Each set as a linked-list, with head and tail, and each node contains value, next node pointer and back-to-representative pointer.
- Example:
- MAKE-SET costs $O(1)$: just create a single element list.
- FIND-SET costs $O(1)$: just return back-to-representative pointer.

Linked-lists for two sets



UNION of
two Sets



UNION Implementation

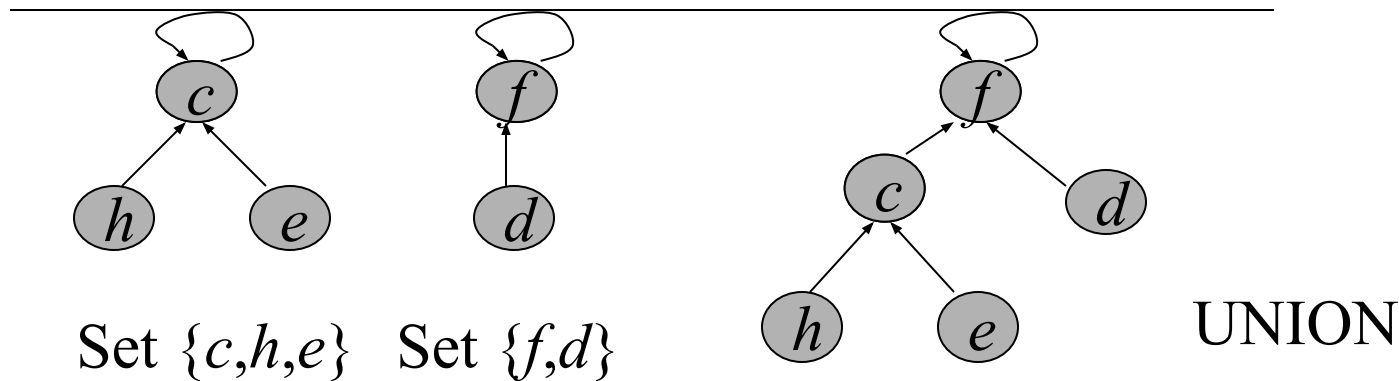
- A simple implementation: $\text{UNION}(x,y)$ just appends x to the end of y , updates all back-to-representative pointers in x to the head of y .
- Each UNION takes time linear in the x 's length.
- Suppose n $\text{MAKE-SET}(x_i)$ operations ($O(1)$ each) followed by $n-1$ UNION
 - $\text{UNION}(x_1, x_2), O(1),$
 - $\text{UNION}(x_2, x_3), O(2),$
 -
 - $\text{UNION}(x_{n-1}, x_n), O(n-1)$
- The UNIONs cost $1+2+\dots+n-1=\Theta(n^2)$
- So $2n-1$ operations cost $\Theta(n^2)$, average $\Theta(n)$ each.
- Not good!! How to solve it ???

Weighted-Union Heuristic

- Instead appending x to y , appending the shorter list to the longer list.
- Associated a length with each list, which indicates how many elements in the list.
- Result: a sequence of m MAKE-SET, UNION, FIND-SET operations, n of which are MAKE-SET operations, the running time is $O(m+n \lg n)$. Why???
- Hints: Count the number of updates to back-to-representative pointer for any x in a set of n elements. Consider that each time, the UNION will at least double the length of united set, it will take at most $\lg n$ UNIONS to unite n elements. So each x 's back-to-representative pointer can be updated at most $\lg n$ times.

Disjoint-set Implementation: Forests

- Rooted trees, each tree is a set, root is the representative. Each node points to its parent. Root points to itself.



Straightforward Solution

- Three operations
 - MAKE-SET(x): create a tree containing x . $O(1)$
 - FIND-SET(x): follow the chain of parent pointers until to the root. $O(\text{height of } x\text{'s tree})$
 - UNION(x, y): let the root of one tree point to the root of the other. $O(1)$
- It is possible that $n-1$ UNIONs results in a tree of height $n-1$. (just a linear chain of n nodes).
- So n FIND-SET operations will cost $O(n^2)$.

Union by Rank & Path Compression

- Union by Rank: Each node is associated with a rank, which is the upper bound on the height of the node (i.e., the height of subtree rooted at the node), then when UNION, let the root with smaller rank point to the root with larger rank.
- Path Compression: used in FIND-SET(x) operation, make each node in the path from x to the root directly point to the root. Thus reduce the tree height.

Path Compression

